AttestRAG: End-to-End Attested Retrieval-Augmented Generation across CPU and GPU TEEs

Draft v0.1, 2026-10-21. Wei Wang. Status markers: **[done]** = implemented and measured on hardware; **[design]** = designed, not implemented; **[TODO]** = open item for the next draft.

Abstract

Enterprises hand contracts, medical records and source code to retrieval-augmented generation (RAG) services, yet every deployed "confidential inference" offering protects only the model call: documents are parsed, embedded, indexed and retrieved in the clear on the provider's host, and embeddings are not a safe intermediate form, since inversion attacks recover the text behind them. We present AttestRAG, a RAG system in which the whole pipeline, from parsing to generation, runs inside hardware-isolated confidential VMs, and in which a client can verify the entire chain before it uploads a single byte. AttestRAG binds the CPU TEE report, the GPU attestation and the measurement of every pipeline component into one composite statement; releases the keys that protect the persisted vector index only after a small, pure policy function accepts that statement; and seals every answer with a one-time key that only the attested node can learn, so that neither the host nor the network can forge or swap a reply. We built AttestRAG on AMD SEV-SNP with a reproducible, measured boot chain (firmware, kernel, initrd, command line and a dm-verity root filesystem under one launch measurement) and evaluated it end to end on the RGB benchmark inside a 32-vCPU confidential VM. Attestation is not the cost: establishing a verified session takes 10–30 ms, two orders of magnitude below a single answer. Retrieval inside the TEE loses nothing (hit@4 = 1.0 on both languages), while CPU-bound generation sets the latency floor (1.4 s per English answer, 3.2 s per Chinese answer) and collapses under concurrency, which is the direct motivation for the GPU TEE stage. The verifier, policy engine and key-release protocol are 2.3 k lines of Python with no dependency beyond a cryptography library, small enough that the release protocol can be analysed as a whole and the policy function run as an executable specification.

1 Introduction

RAG has become the default way to put private corpora in front of a large language model. The corpus is parsed into chunks, each chunk is embedded into a vector, the vectors are indexed, and at query time the nearest chunks are retrieved and pasted into the model prompt. Every one of these stages

sees the customer's data in plaintext, and every one of them typically runs on infrastructure the customer does not control.

The industry's answer has been confidential inference: run the model inside a trusted execution environment (TEE), attest it, and call it over an encrypted channel. This protects exactly one stage. The parser, the embedding model, the vector database and the retriever remain outside the enclave, on the provider's host, where the host operating system, the hypervisor and the operator can read them. Nor is the vector representation a form of protection: few-shot embedding-inversion attacks such as ALGEN reconstruct the original text from a handful of vectors [15]. Cryptographic RAG proposals go the other way: RemoteRAG [13] hides the user's query behind differential-privacy noise and partially homomorphic distance computation, and SAG [14] encrypts chunks and vectors before they are stored, with a formal proof. Neither protects the generation step, where the retrieved plaintext meets the model, and both pay for the cryptography on every query.

Two things make a different design possible now. First, CPU TEEs that isolate a whole virtual machine (AMD SEV-SNP, Intel TDX) are generally available, and a recent evaluation measured LLM inference under TDX at under 10 % overhead, H100 confidential mode at 4–8 %, and a complete RAG pipeline (Elasticsearch, BM25, SBERT) inside TDX at 6–7 % [1]. Second, NVIDIA's Hopper and Blackwell GPUs offer a confidential-computing mode with their own attestation. What is missing is a system that puts the complete pipeline inside these TEEs and, more importantly, lets a client verify the complete pipeline as one object. The gap is not academic: the TEE.fail authors observed that NVIDIA does not bind an H100's attestation to the identity of the VM it serves, so an attacker can forward the GPU report of another machine [7].

This paper presents AttestRAG, a RAG system with the following properties.

- **Everything inside.** Parsing, embedding, the vector index, retrieval and generation run in confidential VMs. The vector database keeps its storage on a tmpfs inside encrypted guest memory; the only thing that reaches the host disk is an authenticated-encrypted snapshot.
- **One composite proof.** The CPU report carries, in its `report_data`, a hash of a session nonce and of an ephemeral public key generated inside the TEE; every GPU report of the session carries a hash of that CPU `report_data`. The launch measurement covers firmware, kernel, initrd, command line and, through a dm-verity root hash on the command line, every byte of the root filesystem, including the models and the vector database binary.
- **Keys only after proof.** A key broker service (KBS) outside the provider's trust domain releases the index key wrapped to the TEE's ephemeral key only after cryptographic verification of the composite evidence and after a policy decision that includes a measurement allow-list, a component-wise minimum TCB and the guest-policy bits of the VM.
- **Client-side verifiability.** The client SDK runs the same verifier and the same policy as the KBS, before it uploads anything. Questions are encrypted to the attested node key and carry a one-time reply key, and the answer is sealed with that key, so an answer the client accepts can only have been produced by the node that could read the question.

We implemented AttestRAG (Section 7) and deployed it on a self-hosted SEV-SNP machine (2x AMD EPYC 9554, 4x H200 NVL) with a reproducible image build whose measurement we verify offline. Section 8 reports the first end-to-end measurements on the RGB benchmark [18]. The GPU stage, the Intel TDX verifier and the formal analysis of the release protocol are designed but not yet implemented; the paper marks these parts explicitly, and Appendix A lists the status of every claim.

Contributions.

1. A threat model and design for a RAG system in which no pipeline stage runs outside a TEE and the cloud operator and the service provider are both untrusted (Sections 4–5). **[done]** for the CPU pipeline.
2. A composite attestation protocol that binds the CPU report, the GPU reports and the pipeline measurement into one verifiable statement, closing the GPU-report-forwarding gap (Section 5.2). **[done]** for the CPU half and the binding rule; **[design]** for the GPU half.
3. Post-attestation key release with a measurement allow-list, component-wise minimum TCB and guest-policy checks, and an index that is sealed per tenant and never present in plaintext on the host (Section 5.3–5.4). **[done]**; rollback protection **[design]**.
4. A key-release protocol and policy designed for formal analysis: the protocol properties that matter (a key reaches only the TEE that proved possession in the same session; no replay; no cross-session GPU evidence) are stated for a symbolic protocol model, and the policy function is small, total and pure so that its executable specification can be the engine that actually runs (Section 6). **[done]** for the code; the protocol model and the executable specification are **[TODO]**.
5. An open-source implementation and an end-to-end evaluation on real SEV-SNP hardware, including the attestation and key-release costs, retrieval and answer quality inside the TEE, and the concurrency behaviour of CPU-only inference (Sections 7–8). **[done]** for the CPU configuration.

2 Background

VM-level TEEs. AMD SEV-SNP and Intel TDX isolate an entire guest VM from the hypervisor: guest memory is encrypted with a per-VM key, integrity is protected (reverse map table on SNP, TD-owner bits on TDX), and the platform security processor signs an attestation report on request. An SNP report (version 5 on current firmware) contains the launch measurement, a 64-byte `report_data` field chosen by the guest, the guest policy bits (debug, migration agent, SMT, …), the current and reported TCB (bootloader, TEE, SNP firmware and microcode security version numbers), the chip identifier and a signature by the chip's VCEK, whose certificate chains to AMD's ASK and ARK. The launch measurement is computed over the initial memory image: with OVMF and the `kernel-hashes` feature it covers the firmware, the kernel, the initrd and the kernel command line [20].

GPU confidential computing. NVIDIA Hopper and Blackwell GPUs can run in a confidential mode in which a single GPU is passed through to a confidential VM, the PCIe link is protected by an SPDM session, device memory is encrypted and the GPU produces an attestation report over its firmware

measurements, verifiable against NVIDIA's attestation service [21]. A reverse-engineering study of this mode found that over a thousand BAR0 register fields remain host-readable and that RPC metadata is in plaintext with timing side channels [12]. The report does not by itself say which VM the GPU is attached to; binding it to the CPU report is the system's job (Section 5.2).

RAG pipeline and what each stage exposes. Ingest parses documents into chunks (plaintext documents). Embedding turns chunks into vectors (plaintext chunks in, invertible vectors out). The vector index stores vectors and the chunk text as payload (both). Retrieval embeds the query and runs nearest-neighbour search (the query, the access pattern over the index). Generation puts the query and the retrieved chunks into one prompt (everything). A design that protects only generation therefore still leaks the corpus, the index and the queries.

Remote attestation and key brokers. The IETF RATS architecture [19] separates the attester, the verifier and the relying party. In confidential-container deployments the relying party is a key broker service (Trustee KBS in Confidential Containers [22]) that releases secrets only to attested workloads. AttestRAG follows this structure with its own small reference KBS, so that the complete release path can be read and, later, formalized.

3 Motivation and problem analysis

Where inference-only TEEs leak. Consider the common deployment: a vector database and an embedding service on ordinary hosts, and a model served from a confidential VM behind an attested TLS endpoint. The host of the vector database sees every chunk in plaintext as payload and every query as a vector and as text (the embedding service receives the raw query). The embedding host sees every document. Only the prompt-to-answer leg is protected, and that leg contains nothing the earlier legs have not already exposed. Encrypting the index at rest with a key held by the database host changes nothing, since the host holds the key.

GPU report forwarding. A GPU attestation report proves that some GPU with the stated firmware is in confidential mode. If the relying party accepts it alongside a CPU report without a cryptographic link between the two, an operator can run the model on an ordinary GPU and present the report of a confidential GPU elsewhere. This is exactly the gap TEE.fail reported for H100 [7]. **[TODO]**: reproduce it with two machines once the GPU stage exists; for now the paper argues it from the report formats.

Why not cryptographic RAG. Differentially private and homomorphic retrieval [13] protects the query against the index host, and encrypted storage [14] protects the index at rest, but both still deliver plaintext chunks to a generation step that must see them; the generator is then the weak point, and the per-query cryptographic cost grows with the index. A TEE design pays a near-constant overhead instead and protects the generator too.

4 Threat model and security goals

Assets. Customer documents, chunks, embeddings, the vector index, queries, answers, and the data-encryption key (DEK) that seals the index.

Trusted. The CPU TEE hardware and firmware at or above the minimum TCB named in the policy (the AMD secure processor and SEV firmware; the TDX module once supported); the GPU confidential-computing firmware when a GPU is used; the measured AttestRAG VM image; the KBS, operated by the customer or a party the customer trusts; and the client SDK.

Untrusted. The cloud provider's host OS, hypervisor and operators; storage and the network; other tenants; and the AttestRAG operator when it is not the customer. The adversary can read, modify and replay anything on disk or on the network; boot a modified image; enable debug; run on an outdated, vulnerable TCB; and replay old evidence or splice evidence from a different VM, session or GPU.

Goals. (G1) Confidentiality of documents, vectors, queries and answers against everything untrusted. (G2) Integrity of retrieval results and answers: a client accepts an answer only if it was produced by the attested node over the question the client sent. (G3) Client-side verifiability: a client can establish G1 and G2 before uploading, with no trust in the provider.

Out of scope. Physical attacks on memory, such as the DDR5 interposer of TEE.fail [7] and the sub-$50 DDR4 interposer of Battering RAM [8], are excluded by the hardware vendors' threat model and by ours; firmware attacks on the AMD secure processor [9] and the SEV-SNP memory-consistency attacks of [10, 11] are the reason the policy carries a minimum TCB; microarchitectural side channels and the access pattern of nearest-neighbour search over the index are only partly mitigated and are discussed in Section 9; availability is not a goal. What the model says is not a security property: documents can carry prompt injections and a small model will follow them; AttestRAG grounds answers in retrieved chunks and citations but does not make the model honest.

5 System design

5.1 Architecture

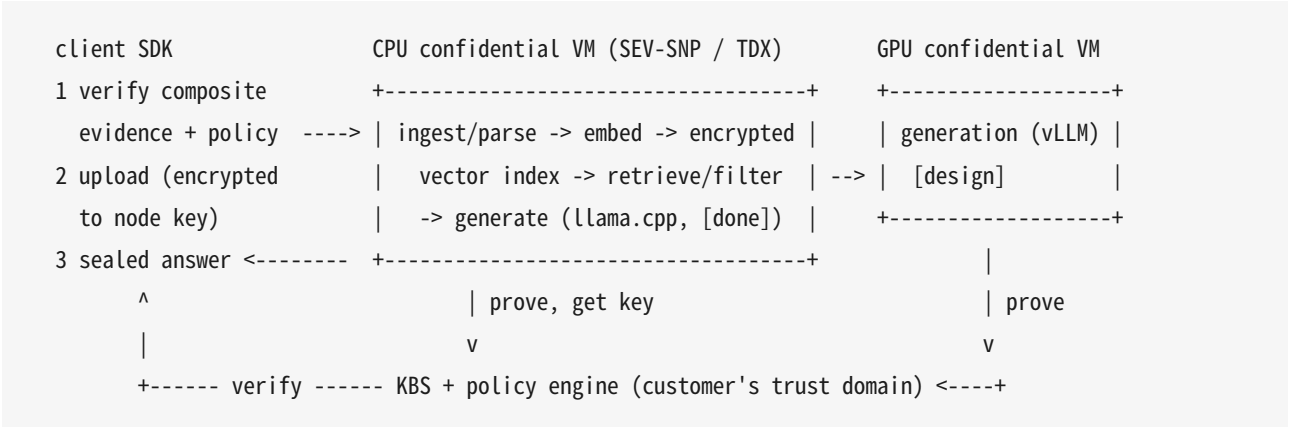


Figure 1. One composite proof covers the whole pipeline. Both confidential VMs must prove themselves to the KBS before they receive any key; the client verifies the same composite evidence before it uploads. In the current implementation generation runs on the CPU inside the same VM; the GPU VM is the next stage.

The CPU confidential VM runs the ingest parser, the embedding model, the vector database (Qdrant) with its storage on a tmpfs inside guest memory, the retriever and, in the current implementation, the generator (a small instruction-tuned model served by llama.cpp). All of these are files in a read-only root filesystem whose dm-verity root hash is part of the launch measurement (Section 5.5), so replacing the embedding model, the vector database binary or the node's Python code changes the measurement and the KBS refuses the key. The node exposes three operations over JSON/HTTP: `attestation(nonce)`, `ingest(doc_id, wrapped_document)` and `query(wrapped_question)`.

5.2 Composite attestation

Every session starts with a 32-byte nonce from the verifier (the KBS, or the client). The node generates an ephemeral X25519 key pair inside the TEE and asks the platform for a report over

```
cpu_report_data = SHA-384("attestrag-rd-v1" ‖ nonce ‖ tee_pubkey)
gpu_report_data = SHA-384("attestrag-gpu-bind-v1" ‖ cpu_report_data)
```

with length-prefixed concatenation, so that no choice of a shorter nonce and a longer key can collide with another pair. `report_data` is the 64-byte field of the report that the guest chooses (offset 0x50 of an SNP report; a SHA-384 digest fills 48 bytes); the platform signs it together with the measurement and the TCB, so whatever is written there is bound to this chip and this launch.

The CPU rule binds two things. The nonce is a 32-byte random value from the verifier, single-use and expiring after 60 s: a report whose `report_data` matches it was produced in this session and cannot be a replay. This matters because SNP reports carry no timestamp; freshness rests entirely on the nonce. The public key is generated inside the TEE for this session and its private half never leaves it: the KBS wraps the DEK to that key, so a man-in-the-middle who relays a genuine report cannot open the key it obtains, and one who substitutes its own key breaks the hash and with it the signature. Together the rule states one fact, proved once: this TEE, in this session, holds the private key.

The GPU rule hangs every GPU report of the session off the CPU report: a GPU report carries not an independent nonce but the hash of the CPU `report_data`. The composite verifier therefore verifies the CPU evidence first, derives the expected GPU `report_data` from it, and only then verifies each GPU evidence. A GPU report is valid for exactly one CPU report, hence one session. This is the check that closes the forwarding gap of Section 3: a confidential GPU's report taken from another machine or another session fails the `report_data` comparison, whatever its own signature says. The two rules yield one statement for the policy engine: this CPU TEE (measurement M, TCB T) holds sk in this session, and these GPUs attested for it in the same session; the policy never judges a GPU on its own.

What the binding does not prove: the GPU rule establishes session freshness, not that the GPU is the one physically attached to this VM over an SPDM-protected link. Closing that requires the VM to attest its own GPU session, for example a measured driver placing the SPDM session identity in the CPU report, and is left for the GPU stage. **[done]** for the rule and the composite verifier (exercised with mock GPU evidence in tests); **[design]** for NVIDIA evidence, whose verifier is not yet written.

What the SNP verifier checks. Parse the version-5 report; reject VLEK-signed reports (not yet supported) and unknown signature algorithms; derive the product (Milan, Genoa, Turin) from the CPUID family and model in the report and refuse a mismatch with the configured product; fetch AMD's ARK and ASK for that product from the bundled copies and compare the ARK against a pinned public key; fetch the VCEK for the chip identifier and reported TCB from AMD's key distribution service, caching it on disk; verify the chain ARK → ASK → VCEK and the certificates' validity window; check that the VCEK's hardware-identifier extension equals the report's `CHIP_ID` and that its SPL extensions equal the report's `REPORTED_TCB`; verify the ECDSA P-384 signature over the report; and compare `report_data` against the expected value. The verifier then emits claims: platform, measurement, decoded TCB components, debug flag, guest policy, launch TCB and signer identity.

5.3 Key hierarchy and release

The KBS holds, per tenant, the index DEK. **[design]**: a tenant master key that wraps per-index DEKs, and per-session keys derived for the client channel; the current implementation has one DEK per key identifier. The release protocol is:

1. the node requests a challenge and receives a single-use nonce with a 60 s lifetime;
2. the node generates the ephemeral key pair, obtains the composite evidence over `cpu_report_data(nonce, pub)` and sends `(key_id, nonce, pub, evidence)`;
3. the KBS consumes the nonce (even on failure), verifies the evidence, evaluates the policy, writes an audit record, and only on `allow` returns the DEK wrapped to `pub` with X25519 ECDH, HKDF-SHA256 (salt = both public keys, info = a versioned label and the key identifier) and AES-256-GCM;
4. the node unwraps the DEK in TEE memory and opens the sealed index.

Policy. The verifier establishes that the evidence is genuine; the policy turns that into a decision that the machine is worth trusting. It is the customer's own statement, evaluated independently by the KBS and by the client with the same rules, that only a TEE running the named image, on firmware at or above a named version and without debug enabled, may receive a key. A policy is a plain record: allowed CPU platforms, a measurement allow-list, a component-wise minimum TCB (bootloader, TEE, SNP firmware, microcode), the allowed guest-policy values (for example `0x30000`, which forbids debug and the migration agent and fixes the ABI version), whether debug is allowed, a freshness window, and the same fields for GPUs. Evaluation is a total function from policy and claims to a decision with a list of reasons; `allow` holds if and only if the list is empty. The TCB comparison is component-wise and never lexicographic: a newer microcode does not compensate for an older SNP firmware.

5.4 Persisted index

The vector database runs with its storage directory on a tmpfs, so vectors and chunk text exist only in encrypted guest RAM. After each ingest the node asks the database for a collection snapshot, seals it with the DEK under AES-256-GCM with the format version, the collection name and the vector dimension as associated data, and writes it to an untrusted data disk provided by the host. At start the node unseals the snapshot in memory and restores the collection through the database's upload interface, so plaintext never touches a file. A host that edits, truncates or swaps the file gets an authentication failure and the node refuses to start. **[done]**. The snapshot is whole-collection, so sealing costs O(index) per ingest; incremental sealing or a dm-crypt volume keyed by the KBS is future work. Rollback to an older, validly sealed snapshot is detected only when the KBS also tracks a monotonic index version **[design]**; permission-aware retrieval (filtering by the caller's rights inside the TEE) and multi-tenancy are **[design]**.

5.5 Measured image and boot chain

The guest image is built reproducibly: a golden qcow2 image is shrunk to a minimal ext4 root, a dm-verity hash tree is computed with a fixed salt, and two consecutive builds are byte-identical. The VM boots OVMF with `kernel-hashes` enabled, so the launch measurement signed by the platform covers the firmware, the kernel, the initrd and the command line; the command line carries `roothash=<dm-verity root hash>`, so every block of the root filesystem is transitively under the measurement. Guest writes go to a tmpfs overlay and are lost at reboot; persistent state exists only through the attested KBS and index path. The expected measurement is computed offline with `sev-snp-measure` [25] from the firmware, kernel, initrd, command line, vCPU count and vCPU model, and equals the value the platform signs. The policy JSON is never part of the image, since it would have to contain its own measurement; the verifier side supplies it.

5.6 Client SDK and answer authenticity

The client runs the same verifier and policy as the KBS. `connect()` draws a nonce, requests the node's evidence and its transport public key, verifies the evidence against `cpu_report_data(nonce, node_pub)`, evaluates the policy, and only then remembers `node_pub`; any call before a successful `connect()` is refused locally. Documents are wrapped to `node_pub` with the document identifier as context. A question is wrapped together with a freshly drawn one-time reply key; the node's answer, including the retrieved hits and citations, is sealed under that reply key with a fixed associated-data label. An adversary on the network or on the host can neither read the question nor produce an answer the client will accept, because learning the reply key requires the node's private transport key, which exists only inside the attested VM. An earlier design that returned answers in plaintext was forgeable on the network; this version fixes it.

6 Formal analysis of key release [design]

Where formal methods earn their place here is the release protocol, not the policy function. The policy function is under a hundred lines of set membership and integer comparison; a proof that it satisfies its own rules would mostly restate it. None of the defects found while building the system were in it: the launch-TCB field read at the wrong offset, the certificate parser rejecting AMD's zero serial numbers, and an earlier answer path that a network adversary could forge all sat in the verifier, the parser and the protocol. The analysis is therefore split into two parts with different tools, and the paper states what neither covers.

Protocol properties. The release protocol of Section 5.3 together with the binding rules of Section 5.2 forms a small protocol between a TEE, a KBS and a network adversary who controls the host. The properties the system relies on are protocol properties with an adversary, and they will be stated and checked in a symbolic model (Tamarin or ProVerif):

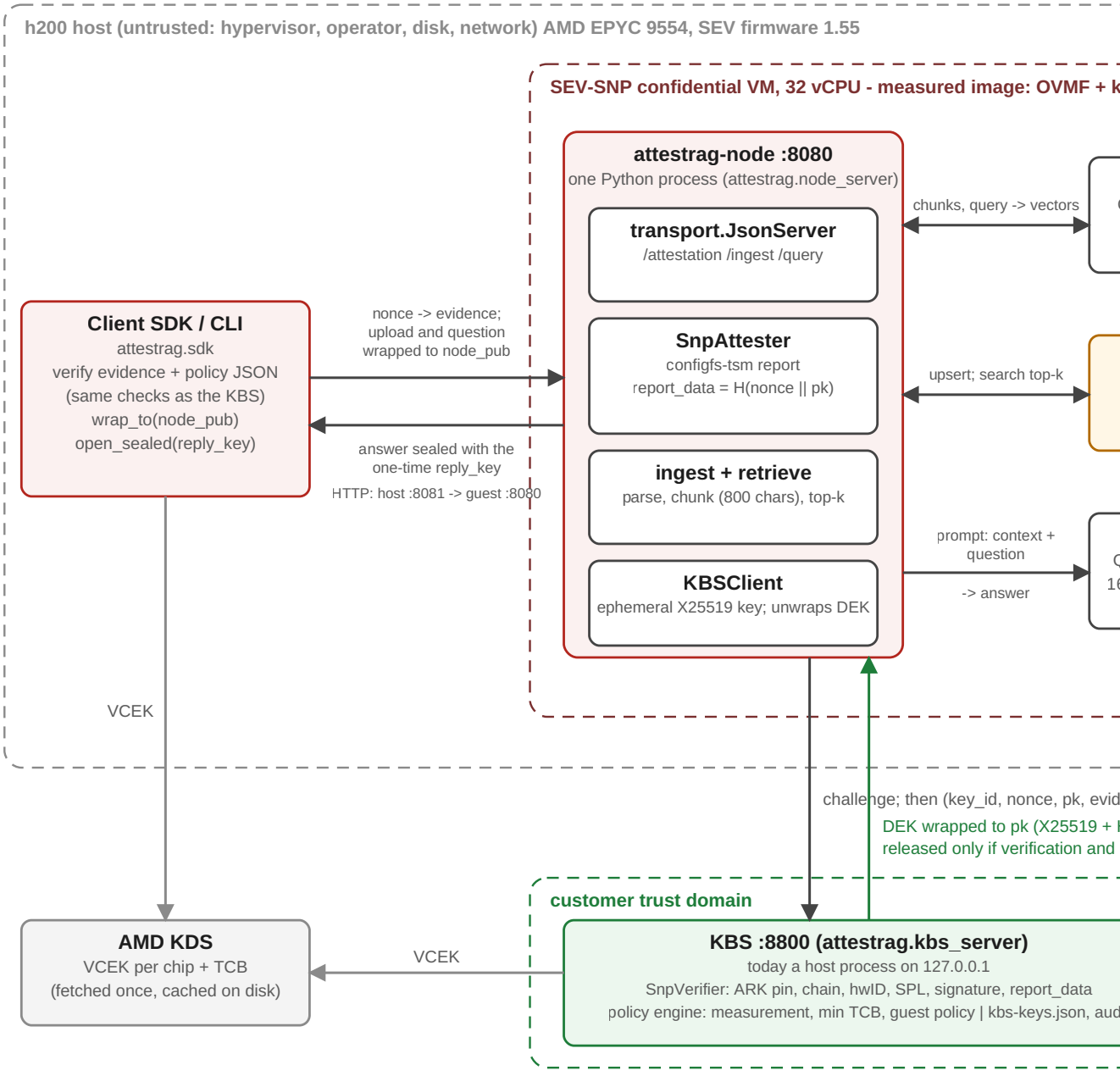
- **Possession**: if the KBS releases a key wrapped to a public key pk, then a TEE that passed verification and policy generated pk and holds the matching private key; an adversary who relays genuine evidence obtains nothing it can open.
- **Freshness**: a release is preceded by a challenge in the same session; evidence from an earlier session, or evidence replayed after its nonce was consumed, is never accepted.
- **No cross-session GPU evidence**: a GPU report accepted with a CPU report was produced over that CPU report's `report_data`; a report from another machine or session is rejected whatever its own signature says. This is the forwarding gap of Section 3, stated as a property.
- **Answer authenticity**: an answer the client accepts was produced by the node that decrypted the question, since the reply key travels only inside the question wrapped to the node's attested key.

The model abstracts the hardware signature as an unforgeable signature by the chip and treats the verifier as correct; it will not find a parsing bug, a wrong reference value in the policy, or a side channel.

Executable policy specification. The policy function, `evaluate(policy, claims, now)`, is total, pure and operates on frozen records of strings and integers, and it is the one piece that must agree exactly between the KBS and every client. It will be written once in Lean 4 with a few algebraic theorems (monotonicity: enlarging an allow-list or lowering a minimum TCB can only turn denials into releases, never the reverse; component-wise TCB: a shortfall in any component implies denial whatever the other components; debug or an unlisted guest policy implies denial), and the usual objection, that the verified model is not the code that runs, is removed by running it: Lean 4 compiles to a native binary, and the KBS and the client SDK will call that binary as the policy engine. Until then the Python function and the Lean function are kept in agreement by differential testing over generated policies and claims (the test suite already enumerates every denial reason). The trusted computing base for release decisions is then the verifier (signature and chain checks, 450 lines of Python), the policy specification and the KBS loop (60 lines).

[TODO]: the symbolic model and its results; the Lean specification with its line counts and proof sizes; the differential-test harness.

7 Implementation



The deployed system on h200: one attested Python process (node, attester, KBS client), three local services inside the measured VM, the sealed snapshot on the untrusted disk, and the KBS in the customer's trust domain.

AttestRAG is about 2,300 lines of Python 3.11 in nine modules (attest, policy, kbs_client, ingest, embed, index, retrieve, generate, sdk) plus the node and the two HTTP services, with 1,100 lines of

tests (61 tests, including a real Genoa report and VCEK as a golden sample verified against AMD's pinned roots) and 600 lines of deployment shell. The only runtime dependencies are `cryptography` and `numpy`. The SNP attester uses the kernel's configfs-tsm interface and needs no vendor tool inside the guest; the verifier bundles AMD's root certificates and fetches VCEKs from AMD KDS with an on-disk cache. The HTTP transport is JSON over the standard library (no TLS yet: metadata such as document identifiers and sizes is visible on the wire). Generation and embedding use llama.cpp's server [23] with Qwen3-Embedding-0.6B (Q8_0, 1024 dimensions) and Qwen2.5-1.5B-Instruct (Q4_K_M); the vector database is Qdrant 1.19 [24] accessed over its REST API. All binaries and model weights are pinned by SHA-256 in the image build.

The self-hosted deployment runs QEMU 11.1 built from source (distribution packages lacked SEV-SNP guest support) with OVMF.amdsev from Ubuntu's 2025.11 build; the Ubuntu 24.04 firmware (2024.02) fails to boot under upstream QEMU SNP. The guest is a 32-vCPU, 16 GB SEV-SNP VM; the vCPU count is part of the measurement. The image is 5.3 GB and boots unattended into attest → key release → serve. An end-to-end script runs 34 checks against the live VM: SNP active, dm-verity root bound to the expected root hash, the platform-signed measurement equal to the offline value, key release allowed under the published policy and denied with a wrong measurement and with a minimum TCB above the platform's, the client refusing to upload to a node whose measurement is not in its policy, and the sealed snapshot restoring the index after a restart.

Two implementation findings are worth recording. The launch-TCB field sits at offset 0x1F0 of a version-5 report, not 0x1F8 as an earlier draft assumed; the real report caught it. And AMD's VCEK certificates carry serial number 0, which recent versions of the `cryptography` library deprecate; the verifier loads them explicitly so that a future hard rejection fails loudly in the golden test rather than silently in production.

8 Evaluation

The evaluation answers three questions: what attestation and key release cost; whether moving the whole pipeline into the TEE loses retrieval or answer quality; and where the time goes. All measurements are from the self-hosted machine (2x EPYC 9554 Genoa, SEV firmware 1.55, 32-vCPU SEV-SNP guest, CPU-only inference) on 2026-10-20, through the attested client path. **[TODO]**: the comparison against an unprotected deployment and against an inference-only TEE deployment, the GPU configuration, larger indexes and a second CPU platform (TDX) are not yet measured; the 10 % end-to-end overhead target of the outline remains a target, not a result.

8.1 Benchmark and method

We use RGB [18], a bilingual benchmark for RAG with 300 English and 300 Chinese news questions, each with positive passages that contain the answer, negative passages on the same topic without the answer, and answer aliases. For 100 sampled questions per language we ingest one document per question holding its positive and negative passages shuffled (median 24 passages); for a held-out 20

% the positives are omitted, so the corpus contains on-topic noise but no answer. Each question is asked once with k = 4. Retrieval is scored as hit@4 (a retrieved chunk shares a 120-character window with a positive passage), answers with RGB's alias match, and rejection by a phrase list in both languages. A concurrency sweep then runs N clients, each with its own attested session, over 16 questions never asked before, since llama.cpp caches prompt prefixes and repeated questions would measure the cache.

8.2 Attestation and key release

Establishing a verified session, which is an SNP report through configfs-tsm, the certificate-chain and signature checks with the cached VCEK, and the policy evaluation, takes 10–30 ms on the client (median 12 ms across 64 sessions in the sweep). The first contact with a chip and TCB adds one fetch from AMD KDS, after which the VCEK is cached. Key release at node start is one such round trip plus the unsealing of the snapshot. Per-user attestation is therefore free compared with one answer.

8.3 Retrieval and answer quality inside the TEE

	Chinese	English
corpus (100 documents)	814 chunks	3,762 chunks
hit@4	1.00	1.00
positive chunks among the 4 (mean)	3.2	3.1
answer accuracy (RGB alias match)	0.94	0.96
false rejection on answerable questions	0.00	0.00
held out: rejection ("context insufficient")	0.05	0.05

Retrieval through the encrypted index and the in-TEE vector database loses nothing: a positive passage is among the four retrieved chunks for every question, in a corpus where two thirds of the passages are on-topic noise. The remaining answer errors are the 1.5B generator picking the wrong fact out of a correct context. The weak point is negative rejection: when the answer is absent from the corpus the model refuses only 5 % of the time and otherwise answers from pretraining or fabricates. This is the behaviour the RGB authors report even for ChatGPT; for AttestRAG it means that the generator, not the TEE, bounds answer trustworthiness, and that a production node needs a larger model or a relevance gate that refuses when no chunk passes a similarity threshold. Rejection rate should be reported as a security-relevant metric alongside accuracy.

8.4 Latency and throughput

single user, client side	Chinese	English
ingest, per document (median 24 passages), p50 / p90	4.5 s / 5.9 s	6.1 s / 8.6 s
ingest throughput	1.8 chunks/s	6.2 chunks/s
answer (embed + search + generate), p50 / p90	3.2 s / 3.7 s	1.4 s / 1.7 s

Chinese is slower because the same 800-character chunk is about three times as many tokens: the prompt is around 2,000 tokens against 700 for English, and CPU prompt processing runs at about 700 tokens per second on 32 vCPUs. Ingest time is embedding plus the whole-collection sealed snapshot after every document, so it grows with the collection. For calibration, the first configuration of the same VM (8 vCPUs, 2 embedding threads, 6 generation threads) ingested 0.31 chunks/s and answered a Chinese question in 10.3 s; the vCPU count and the thread counts are part of the measured image, so the change was a rebuild with a new expected measurement.

users	English q/s	English p50 / p90	Chinese q/s	Chinese p50 / p90	errors
1	0.60	1.4 s / 2.7 s	0.36	3.1 s / 3.5 s	0
2	0.21	8.9 s / 16.4 s	0.18	12.5 s / 16.7 s	0
4	0.23	17.4 s / 25.6 s	0.17	17.6 s / 35.8 s	0
8	0.29	26.5 s / 39.5 s	0.24	34.3 s / 47.6 s	0

Throughput drops three-fold as soon as two users overlap and does not recover: the inference server runs four slots on a unified KV cache and interleaves the prompts, while the embedding server and the generation server, 32 threads each, run at the same time on 32 vCPUs and oversubscribe them. (With the inference server's default 4,096-token cache the same sweep failed outright for two or more users with KV-cache allocation errors; a 16k cache removed the errors but not the slowdown.) None of this is attestation cost; it is CPU inference, and it is the quantitative motivation for the GPU stage: prompt processing at 700 tokens per second is the whole budget, and a GPU in confidential mode does the same work in milliseconds.

8.5 Security checks

The end-to-end suite exercises the negative paths on the live VM: a policy with a different measurement makes the client refuse to upload (exit 3) and makes the KBS deny the key with the reason `measurement not in allow-list`; a minimum SNP firmware version one above the platform's is

denied with `tcb 'snp'=24 < required 25`; a guest-policy allow-list of `0x20000` denies the `0x30000` VM; flipping one bit in the report's policy or measurement field breaks the signature; flipping one bit in the reported TCB breaks the match against the VCEK's SPL extensions; a report for a different nonce fails the `report_data` check; and a sealed snapshot with one byte changed, the wrong key, or the wrong collection name in the associated data fails authentication and the node refuses to start. **[TODO]**: GPU-report forwarding and index rollback, once the GPU stage and the version counter exist.

9 Discussion and limitations

Physical and side-channel attacks. DDR interposer attacks [7, 8] break the hardware assumption of every VM-level TEE and are out of scope; a platform whose firmware has a published arbitrary-code-execution bug [9] is excluded by the minimum-TCB check only once a fixed firmware exists and the policy names it. Access-pattern leakage of nearest-neighbour search inside the VM (page faults, cache, the ciphertext side channels of [11]) could reveal which chunks a query touches; ORAM-style indexing is a possible mitigation at a cost we have not measured. On the GPU side, the host-readable registers and plaintext RPC metadata documented in [12] remain even when the GPU stage exists.

Single GPU and the GPU stage. The design assumes one GPU passed through to one confidential VM; multi-GPU confidential computing (NVLink, SPDM across devices) is a later tier. When the GPU stage exists, memory swapping under confidential mode will matter: PipeLLM measured up to 88 % throughput loss from encrypting swapped GPU memory and recovered most of it with speculative pipelined encryption [4]. Until the GPU stage exists, generation runs on the CPU, and the numbers in Section 8.4 are a CPU-only baseline rather than the system's target performance.

Key management. The reference KBS keeps keys in memory and in a 0600 JSON file, has one policy for all keys, no rotation or revocation, no durable audit log, no TLS and no client authentication; a deployment would use Trustee or an HSM-backed broker with the same protocol. Freshness relies entirely on the single-use nonce, since SNP reports carry no time. VLEK-signed reports and Intel TDX evidence are not yet verified.

Model behaviour. The TEE guarantees what code ran and what data it saw, not that the model told the truth. Section 8.3 shows a 1.5B model fabricating answers when the corpus has none; prompt injection through documents is likewise outside the TEE's reach.

10 Related work

Confidential inference. Measurements of LLM inference under VM TEEs agree that the model call itself is cheap: under 10 % on TDX and 4–8 % on H100 confidential mode [1], 1–3 % on B200 with TDX when configured correctly and 30–40 % with default settings [3], with a systematic H100 study in [5]. The same ETH Zurich study also ran a RAG pipeline inside TDX at 6–7 % overhead [1], as an evaluation rather than a system: it does not attest the pipeline or bind its components. EnclaveX [2] is the closest

system: TDX confidential VMs with H200 confidential GPUs orchestrated by Kubernetes and SCONE, with keys released only after a unified attestation, at about 1.3 % attestation overhead; it does not cover retrieval, and its attestation is an orchestration-level property rather than something the end client verifies. PipeLLM [4] reduces the cost of encrypting swapped GPU memory under H100 confidential mode. AttestRAG differs in scope (the whole RAG pipeline, including the index and its persistence), in the composite proof, and in the client-side verifiability of the complete chain.

TEE attacks. TEE.fail [7] uses a DDR5 bus interposer costing under $1,000 to extract Intel's provisioning key, forge TDX attestations and steal ECDSA keys from SEV-SNP VMs, and it is also the source of the GPU-report-forwarding observation that Section 5.2 addresses. Battering RAM [8] does the same on DDR4 for under $50 against SGX and patched SEV-SNP platforms. ASPBreaker [9] achieves arbitrary code execution on the AMD secure processor from an old firmware version and shows that the compromise carries over to newer versions, decrypting VM memory and forging reports; Fabricked, StackWarp and STALEUS [10] and Heracles [11] attack SEV-SNP through Infinity Fabric misconfiguration, memory inconsistency and chosen plaintexts. Together they define the boundary of our threat model and motivate the component-wise minimum-TCB check, which is the only defence a relying party has against firmware with known vulnerabilities, and the explicit exclusion of physical access. For the GPU, [12] documents the host-visible state that remains under confidential mode.

Cryptographic and private RAG. RemoteRAG [13] protects the user's query with differential privacy and partially homomorphic encryption, not the document store; SAG [14] encrypts chunks and vectors before storage and proves security formally; ALGEN [15] shows why vectors need protecting at all. Surveys of RAG security [16, 17] catalogue poisoning and injection, which are orthogonal to the confidentiality problem addressed here. Commercial confidential-AI offerings (Opaque, Phala, Fortanix) are not described in enough detail to compare.

11 Conclusion

AttestRAG shows that a RAG pipeline can run entirely inside confidential VMs and be verified as one object by the party whose data it handles. On real SEV-SNP hardware, attestation and key release cost tens of milliseconds per session, retrieval inside the TEE is lossless, and the measured boot chain turns every pipeline component into a policy decision. What remains is the GPU stage and its binding to the VM, the formal analysis of the release protocol, and the comparison against unprotected and inference-only deployments that will decide whether the end-to-end overhead target holds.

References

arXiv entries are preprints without peer review; conference entries are accepted papers. Entries 1–17 follow `draft/confidential-rag-papers.md` (retrieved 2026-10-15).

1. Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs. ETH Zurich. arXiv 2509.18886, 2025. https://arxiv.org/abs/2509.18886
2. EnclaveX: End-to-End Confidential AI with CPU/GPU TEEs. TU Dresden, Scontain, STACKIT. arXiv 2606.31408, 2026. https://arxiv.org/abs/2606.31408
3. Benchmarking Confidential Computing Performance on NVIDIA Blackwell GPUs. arXiv 2608.26575, 2026. https://arxiv.org/abs/2608.26575
4. PipeLLM: Fast and Confidential LLM Services with Speculative Pipelined Encryption. Shanghai Jiao Tong University IPADS. ASPLOS 2025. https://ipads.se.sjtu.edu.cn/zh/publications/TanASPLOS25.pdf
5. Performance of Confidential Computing GPUs. IBM Research. ICDCS 2025. https://arxiv.org/abs/2505.16501
6. GPU Travelling: Efficient Confidential Collaborative Training. CCS 2025. https://gzs715.github.io/pubs/GPUTRAVEL_CCS25.pdf
7. TEE.fail: Breaking TEEs via DDR5 Memory Bus Interposition. Georgia Tech, Purdue. 2025. https://tee.fail/files/paper.pdf
8. Battering RAM: Low-Cost Interposer Attacks on Confidential Computing via Dynamic Memory Aliasing. KU Leuven, University of Birmingham. IEEE S&P 2026. https://batteringram.eu/batteringram.pdf
9. Jailbreaking the AMD Secure Processor (ASPBreaker). Institute of Software, Chinese Academy of Sciences. USENIX Security 2026. https://www.usenix.org/system/files/usenixsecurity26-shen.pdf
10. Fabricked; StackWarp; STALEUS. USENIX Security 2026. https://www.usenix.org/system/files/usenixsecurity26-schlueter-fabricked.pdf , https://www.usenix.org/system/files/conference/usenixsecurity26/sec26_prepub_zhang-ruiyi.pdf , https://www.usenix.org/conference/usenixsecurity26/presentation/schlueter-2
11. Heracles: Chosen Plaintext Attack on AMD SEV-SNP. ETH Zurich. CCS 2025. https://heracles-attack.github.io/Heracles-CCS2025.pdf
12. Blueprint, Bootstrap, and Bridge: A Security Look at NVIDIA GPU Confidential Computing (listed as "NVIDIA GPU Confidential Computing Demystified" in the reading list). IBM Research, Ohio State University. arXiv 2507.02770, 2025. https://arxiv.org/abs/2507.02770
13. RemoteRAG: A Privacy-Preserving LLM Cloud RAG Service. USTC, CUHK. Findings of ACL 2025. https://aclanthology.org/2025.findings-acl.197.pdf
14. Provably Secure Retrieval-Augmented Generation (SAG). BUPT. arXiv 2508.01084, 2025. https://arxiv.org/abs/2508.01084

15. ALGEN: Few-shot Inversion Attacks on Textual Embeddings. ACL 2025. https://aclanthology.org/2025.acl-long.1185.pdf
16. Towards Secure RAG: A Comprehensive Review of Threats, Defenses and Benchmarks. arXiv 2603.21654, 2026. https://arxiv.org/abs/2603.21654
17. Security and Privacy in RAG: Architectures, Threats, Defenses, and Future Directions. arXiv 2606.25533, 2026. https://arxiv.org/abs/2606.25533
18. J. Chen, H. Lin, X. Han, L. Sun. Benchmarking Large Language Models in Retrieval-Augmented Generation. AAAI 2024. https://github.com/chen700564/RGB
19. H. Birkholz et al. Remote ATtestation procedureS (RATS) Architecture. IETF RFC 9334, 2023.
20. AMD. SEV Secure Nested Paging Firmware ABI Specification (report format, launch measurement, TCB versions). [TODO: pin the document revision used]
21. NVIDIA. Confidential Computing for Hopper and Blackwell GPUs: deployment guide and attestation SDK. [TODO: pin the document versions]
22. Confidential Containers project: Trustee key broker service. https://github.com/confidential-containers/trustee
23. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp (build b11753)
24. Qdrant vector database, version 1.19. https://github.com/qdrant/qdrant
25. sev-snp-measure: calculate AMD SEV-SNP launch measurements. https://github.com/virtee/sev-snp-measure

Appendix A: claim status

Claim or component	Status	Evidence
Whole CPU pipeline inside a measured SEV-SNP VM	done	deploy/h200, e2e 34/34
SNP verifier (chain, hwID, SPL, signature, report_data)	done	tests with a real Genoa report
Composite CPU + GPU binding rule and verifier	done (rule), design (NVIDIA evidence)	mock GPU tests
Key release after proof; measurement, min-TCB, guest-policy checks	done	KBS audit, e2e deny cases
Index sealed with AEAD, never plaintext on host	done	Qdrant on tmpfs + sealed snapshots
Index rollback protection	design	needs KBS version counter
Client verifies before upload; sealed answers	done	SDK, e2e
Multi-tenant keys, permission-aware retrieval	design	
Intel TDX verifier, VLEK reports	TODO	
GPU confidential VM for generation	design	host has 4x H200 NVL; CC mode not yet enabled
Symbolic model of the release protocol (Tamarin/ProVerif)	TODO	properties stated in Section 6
Executable policy specification (Lean 4) + differential tests	TODO	engine written to be formalizable
Comparison against unprotected and inference-only baselines	TODO	
RGB end-to-end numbers on SEV-SNP, CPU-only	done	docs/benchmarks.md